Zero-Copy Data Sharing Between Python and Rust

with PyO3 and Apache Arrow



Allen Y · Associate Software Engineer, Red Hat

PyCon ES 2026 · 8 Nov · Aula B7

Slides and code: **github.com/<your-handle>/pycones26-arrow-pyo3**

THE PROBLEM

A story you may recognise

Your Python pipeline is too slow.

You profile. You rewrite the hot function in Rust.

In isolation it is **40× faster**.

End to end, the pipeline is **slower than before**.

The border

was the bottleneck — not the computation

Where we're going

01 The naive bridge — PyO3, and what it costs

02 The serialization trap — “just use JSON”

03 Arrow as shared memory — zero copies

04 Pitfalls and production

01
The naive bridge

PyO3, and what it costs

PyO3: a native module

```rust
#[pyfunction]
fn add(a: i64, b: i64) -> i64 { a + b }

#[pymodule]
fn demo(m: &Bound<PyModule>) -> PyResult<()> {
    m.add_function(wrap_pyfunction!(add, m)?)
}
```

Three annotations. That is the whole bridge.

Build it, call it

```
$ maturin develop --release

>>> import demo
>>> demo.add(2, 3)
5
```

Scalars are easy.

Now pass five million of them.

The obvious way

```rust
#[pyfunction]
fn sum_list(values: Vec<f64>) → f64 {
    values.iter().sum()
}
```

```
>>> sum_list(data)    # data: 5,000,000 floats
```

It looks like a function call. It is a full copy.

01 · MEASURED

What that costs

461 ms
Arrow → list → Rust
5,000,000 float64 · 40.0 MB

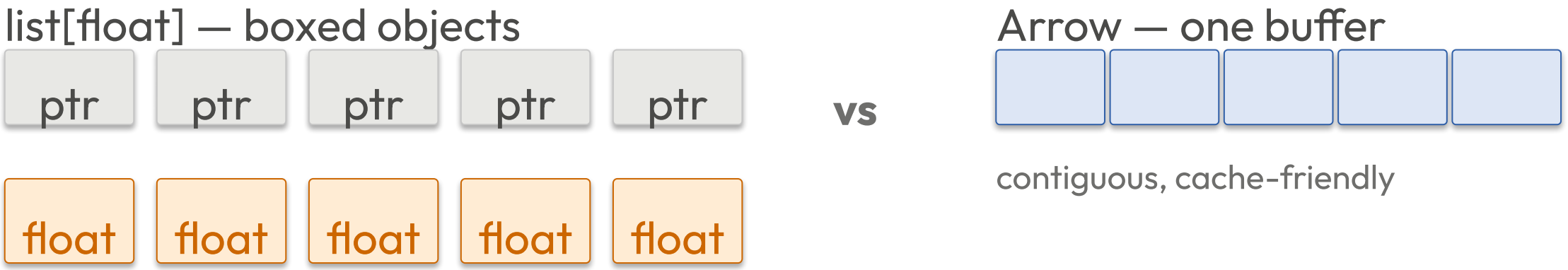
397 ms
of it: .to_pylist()
format conversion

83 ms
of it: the PyO3 call
unboxing 5M PyFloats

Not one byte of this is arithmetic.

Every element becomes a Python object, then gets unboxed again.

Why a Python list is expensive



24 bytes of object header per 8 bytes of payload.

Pointer chasing defeats the cache — the CPU waits on memory.

02
The serialization trap

“just send it as JSON”

The microservices habit

```
#[pyfunction]
fn sum_json(payload: &str) → PyResult<f64> {
    let v: Vec<f64> = serde_json::from_str(payload)?;
    Ok(v.iter().sum())
}
```

40.0 MB of float64 becomes a 101.3 MB string.

Then it is parsed back into floats.

What that costs

6.0 s

to sum five million numbers

5369 ms in json.dumps · 281 ms parsing in Rust

03
Arrow as shared memory

stop moving data; share it

Don't move it.

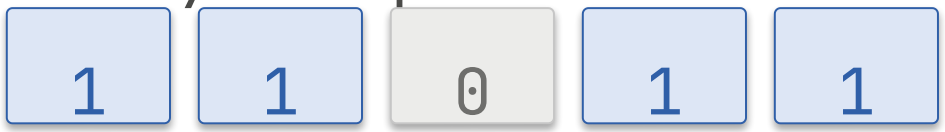
Both languages can point at the same bytes.

Arrow is a memory layout, not a library

Columnar	one column, laid out contiguously
Specified	the byte layout is the standard
Language-neutral	Rust, Python, C++, Java read the same bytes
Null-aware	a validity bitmap: one bit per value

A float64 column, completely

validity bitmap



values buffer



Two flat buffers.

No headers, no pointers,

no objects.

Sharing the column = handing over two pointers and a length.

The Arrow C Data Interface

```
struct ArrowArray {
  int64_t length;
  const void** buffers;      // the pointers
  void (*release)(struct ArrowArray*);
};
```

The capsule transfers a pointer and ownership.

release() keeps the buffer alive while Rust reads.

One dunder, every library

```
>>> obj.__arrow_c_array__()
(<PyCapsule schema>, <PyCapsule array>)
```

pyarrow · polars · pandas · duckdb · nanoarrow

all speak it, in both directions.

The Rust side

```
#[pyfunction]
fn sum_arrow(arr: PyArray) → PyResult<Option<f64>> {
    let (arr, _f) = arr.into_inner();   // zero copy
    let col = arr.as_any()
        .downcast_ref::<Float64Array>()?;
    Ok(aggregate::sum(col))                  // null-aware
}
```

No .to_pylist(). No serde. No allocation.

The Python side

```
import pyarrow.parquet as pq
import arrowbridge

t = pq.read_table("events.parquet")    # Arrow
col = t.column("value").combine_chunks()
arrowbridge.sum_arrow(col)
```

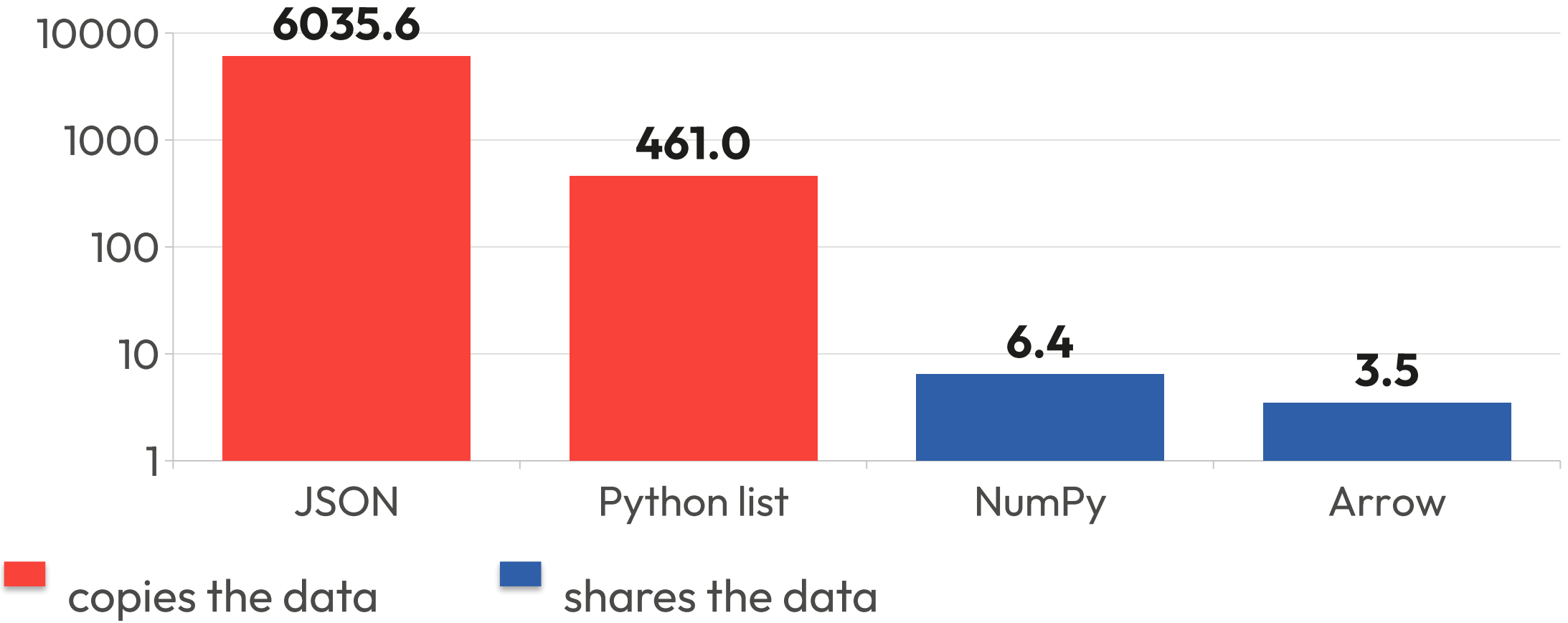
That is the whole integration.

Proof: the same memory

```
>>> arr.buffers()[1].address
990584432
>>> arrowbridge.arrow_data_ptr(arr)
990584432
```

Identical. Nothing was copied or serialized.

Same answer, four ways



milliseconds · 5,000,000 float64 (40.0 MB) · median of 7 runs · Intel(R) Core(TM) Ultra 7 165U

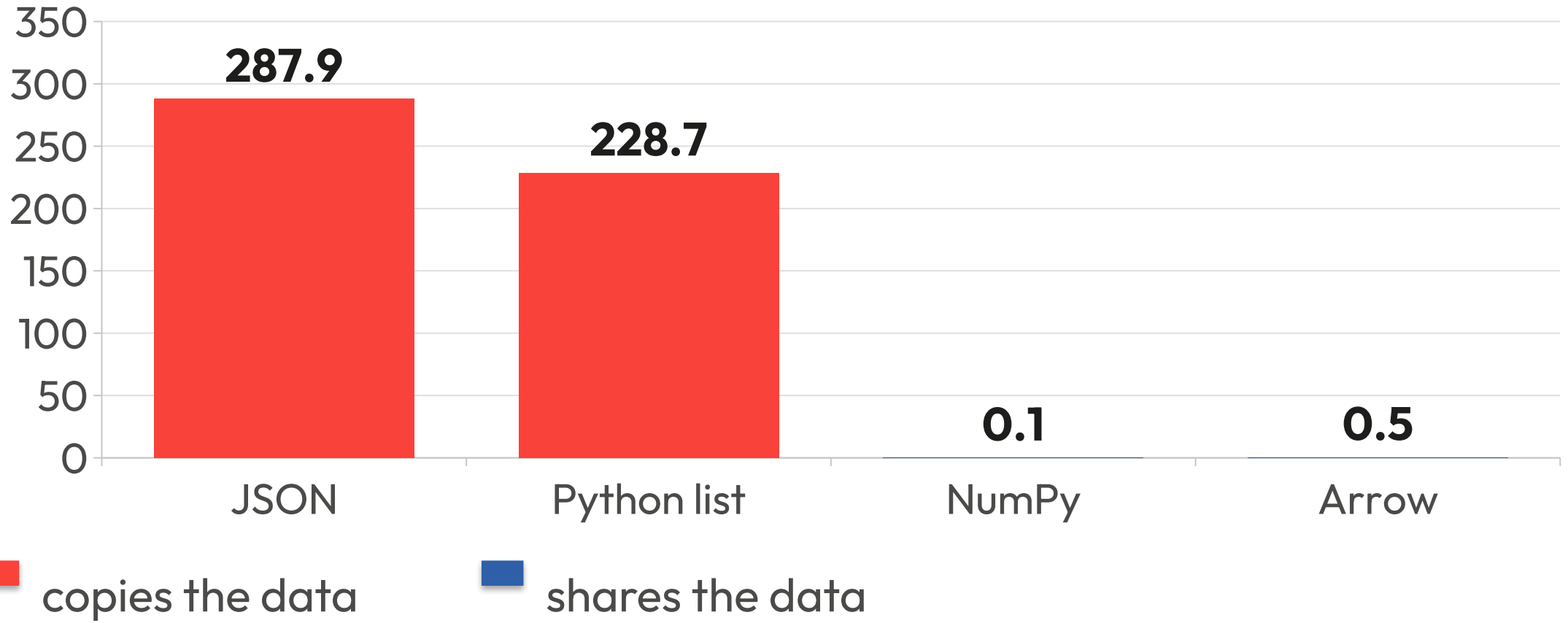
Against the baseline

132×
faster than a Python list

1730×
faster than JSON

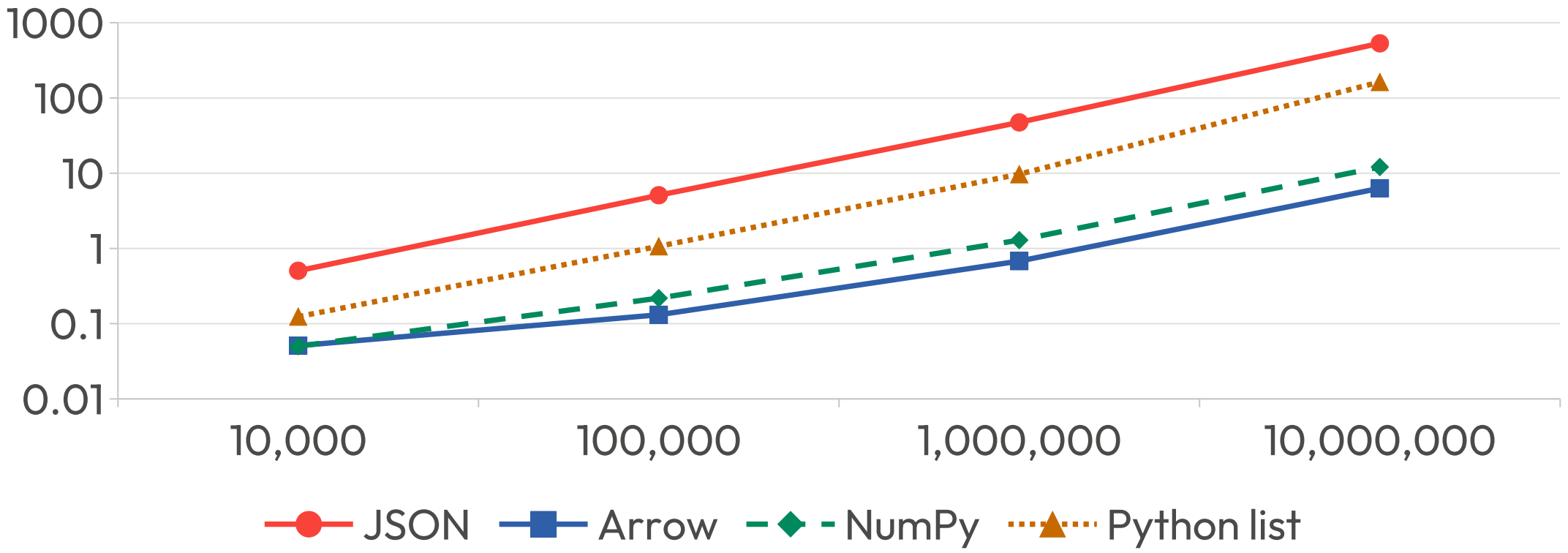
3.5 ms vs 461 ms vs 6036 ms

Peak memory on top of the data



megabytes of extra peak RSS beyond the 40.0 MB Arrow array

How it scales



milliseconds (log) vs row count · flatter is better

“NumPy was zero-copy — why Arrow?”

	NumPy	Arrow
Numeric arrays	yes	yes
Null values	no	yes
Strings	no	yes
Nested types	no	yes
Whole tables	no	yes

Same speed. Far wider contract.

03 · MEASURED

Strings: the buffer protocol gives up

136 ms
list[str] → Vec<String>
a String allocated per row

12 ms
Arrow StringArray
borrowed &str, no allocation

11× faster on 1,000,000 strings.

Arrow packs strings into one buffer plus offsets;

Rust borrows slices out of it.

Nulls come for free

```
>>> a = pa.array([1.0, None, 3.0, None, 6.0])
>>> arrowbridge.sum_arrow(a)
10.0
```

The validity bitmap travels with the buffer.

Rust's kernels are null-aware. Nothing to translate.

Whole tables, too

```
>>> batch = pa.record_batch(columns)
>>> arrowbridge.batch_shape(batch)
(4, 2)
```

A RecordBatch carries the schema.

Column names, types and nullability all survive.

04
Pitfalls and production

what I wish I'd known

Pitfall 1: two arrow versions

```
error[E0308]: mismatched types
  expected `arrow_schema::DataType`
  found    `DataType`
note: multiple versions of crate `arrow_schema`
```

Cargo links arrow 59 and 60 side by side.

Pin arrow to what pyo3-arrow depends on.

Find duplicates with: cargo tree -d

Pitfall 2: ChunkedArray is not an Array

```
>>> table.column("value")
<ChunkedArray ... 12 chunks>

>>> table.column("value").combine_chunks()
# one buffer -- and this COPIES
```

Budget for that copy, or accept a

RecordBatchReader and stream the chunks.

Pitfall 3: ownership and the GIL

```
#[pyfunction]
fn heavy(py: Python, arr: PyArray) → PyResult<f64> {
    let (arr, _) = arr.into_inner();
    py.detach(|| expensive_scan(&arr))   // GIL off
}
```

Never cache a raw pointer past the call.

Releasing the GIL is what makes Rust pay off.

Pitfall 4: doing this when you shouldn't

0.12 vs 0.05 ms
at 10,000 rows
not worth a rewrite

461 vs 3.5 ms
at 5,000,000 rows
now it dominates

Reach for this when the data is already columnar,

you cross the boundary often, or memory is tight.

Otherwise a Python list is fine.

How to tell it is really zero copy

Compare pointers	assert the addresses match, in a test
Watch peak RSS	VmHWM in /proc — a copy shows as a step
cargo tree -d	duplicate crates cause baffling errors
Count the chunks	more than one means combine_chunks copies

You are already using this

Polars
Rust engine, Arrow memory

DataFusion
SQL engine in Rust

delta-rs
Delta Lake, Rust core

pydantic-core
PyO3 at the heart of v2

Arrow is the substrate. The C Data Interface is the plug.

Three things to take home

01 **The boundary is the bottleneck**

Profile the crossing before you rewrite anything.

02 **Arrow is a layout, not a library**

Agree on bytes and the copy disappears.

03 **Zero copy is testable**

Assert pointer equality. Don't assume; measure.

Gracias · Thank you

Questions?

Slides, code and benchmarks

github.com/<your-handle>/pycones26-arrow-pyo3

LinkedIn

linkedin.com/in/allen-yesudasan

Scan for LinkedIn